
Austrian ICD-10 Educational Prototype

User Guide

Installation, learner workflow, maintenance, and troubleshooting

At a glance

This guide is for people who want to install and use the working thesis prototype. The quickest route uses Docker Compose and the project's published container images. No local PHP, MySQL, Node.js, Python, or Composer installation is required.

Guide revision	0.1
Last updated	8 August 2026
Prototype data	BMASGPK 2026 working baseline
Repository	github.com/junomarx/bsc-thesis-icd10

Contents

1 Quick introduction	1
2 Installation	1
2.1 What you need	1
2.2 Install and start with the published images	1
2.3 Use a different local port	2
2.4 Build locally instead of pulling images	2
3 Using the prototype	2
3.1 Choose a case	2
3.2 Review the case facts	3
3.3 Search for and submit a code	3
3.4 Interpret the feedback	3
4 Starting, stopping, and updating	3
4.1 Check status and logs	3
4.2 Stop and restart	4
4.3 Update to the newest repository and images	4
4.4 Reset the prototype data	4
5 Optional: run the automated tests	4
6 Troubleshooting	5
6.1 Docker cannot connect	5
6.2 Port 8080 is already allocated	5
6.3 The database is unhealthy after an update	5
6.4 The page does not load or cases do not appear	5
6.5 A published image is unavailable on this computer	5
6.6 The browser tests cannot start	5
7 Data, privacy, and scope	5
8 Further project documentation	6
A Command summary	6

1 Quick introduction

Austrian ICD-10 Educational Prototype is a bachelor-thesis software artefact for practising a small, explicitly defined subset of Austrian ICD-10 coding. It presents synthetic teaching cases, lets a learner select one of the supported codes, and evaluates that answer with deterministic rules. Feedback is returned as **Correct**, **Suboptimal**, or **Incorrect**, together with an explanation and, where applicable, a suggested improvement.

The prototype is deliberately small. It is intended to demonstrate and test the project's rule-based educational approach, not to provide a complete coding platform.

Educational use only

The cases are synthetic. The prototype does not diagnose patients, provide clinical decision support, perform official coding, or support reporting or reimbursement. Do not enter real patient data.

2 Installation

2.1 What you need

Before starting, install and start the following:

- **Docker Desktop** on macOS or Windows, or Docker Engine on Linux;
- **Docker Compose v2**, available through the `docker compose` command;
- **Git** to download and update the repository;
- a modern web browser; and
- an internet connection for the initial download of the repository and container images.

Check Docker before continuing:

```
docker --version
docker compose version
```

Both commands should print version information. If Docker reports that it cannot connect to the daemon, open Docker Desktop or start the Docker service.

2.2 Install and start with the published images

Open a terminal and run:

```
git clone https://github.com/junomax/bsc-thesis-icd10.git
cd bsc-thesis-icd10
docker compose pull
docker compose up -d --wait app
```

The first start downloads several images and may take a few minutes. Compose then starts MySQL, loads the versioned prototype baseline, and starts the web application in the correct order.

Open <http://localhost:8080> in a browser.

Optionally verify the HTTP endpoint from a terminal:

```
curl http://127.0.0.1:8080/api/health
```

The expected response is:

```
{"status":"ok"}
```

What was installed?

Docker created containers for the database, one-time baseline loader, and application, plus a named database volume. The source checkout remains a normal folder and can be updated with Git. Learner attempts are not stored.

2.3 Use a different local port

Port 8080 is the default. If it is already in use, stop the stack and choose another port. On macOS or Linux:

```
docker compose down
APP_HTTP_PORT=8081 docker compose up -d --wait app
```

In Windows PowerShell:

```
docker compose down
$env:APP_HTTP_PORT = "8081"
docker compose up -d --wait app
```

Then open <http://localhost:8081>. The selected environment value remains in the current terminal session; clear or change it before a later start if you want to return to port 8080.

2.4 Build locally instead of pulling images

If a published image is unavailable for the host architecture, or if you want to run the exact source in the checkout, build the images locally:

```
docker compose build
docker compose up -d --wait app
```

This takes longer on the first run because Docker builds the frontend, backend, and baseline-loader images. It still does not require the language toolchains to be installed directly on the host.

3 Using the prototype

3.1 Choose a case

The first screen shows the educational-use disclaimer and the available synthetic cases. Each case button identifies the case, encounter setting, and diagnosis role. Select a case to begin.

3.2 Review the case facts

The case screen shows only facts that are relevant to the implemented rule set, such as:

- the encounter setting;
- whether the code is being considered as a main or secondary diagnosis; and
- stable-phase FEV1 as a percentage of predicted, when that fact applies.

This concise presentation is intentional. The prototype is a rule-focused demonstrator rather than a full clinical vignette system.

3.3 Search for and submit a code

Use **Search supported codes** to filter by ICD-10 code, full designation, or short designation. Select one result using its radio button, then choose **Submit code**. The button remains disabled until a code is selected.

Only codes in the case's declared response domain are offered. The list is not a complete Austrian ICD-10 catalogue, and free-text or arbitrary code submission is not part of the learner interface.

3.4 Interpret the feedback

Result	Meaning in this prototype
Correct	The submitted code is a declared acceptable response for the case.
Suboptimal	The answer is usable within the implemented rules, but the case supports a more specific code. A suggested improvement is shown.
Incorrect	A hard rule was violated, for example a status restriction, required coding depth, or conflict with the stated case evidence.
Not evaluated	The request could not be classified because a required case or code relationship was unavailable. This is normally a data or application problem, not an expected learner outcome.

The explanation beneath the result states the rule-relevant reason. Choose **Try another code** to answer the same case again, or **Back to cases** to choose a different case.

4 Starting, stopping, and updating

Run all commands in the repository folder containing `docker-compose.yml`.

4.1 Check status and logs

```
docker compose ps
docker compose logs app
docker compose logs db
docker compose logs bootstrap
```

Add `-f` to a log command to follow new output, for example `docker compose logs -f app`. Press Ctrl+C to stop following the log; this does not stop the container.

4.2 Stop and restart

Stop the containers while keeping the database volume:

```
docker compose down
```

Start the prototype again later:

```
docker compose up -d --wait app
```

4.3 Update to the newest repository and images

```
git pull --ff-only
docker compose pull
docker compose up -d --wait app
```

The bootstrap service checks the versioned baseline during startup and is safe to run again when the baseline is unchanged.

4.4 Reset the prototype data

This removes the database volume

The following command deletes the local Compose database volume. In the current prototype that volume contains only the reproducible, versioned baseline—not learner accounts or attempt histories. Do not reuse this command unchanged if persistent user data is added in a future version.

```
docker compose down -v
docker compose up -d --wait app
```

Resetting is useful after an incompatible `mysql:latest` upgrade or when you need a clean copy of the baseline.

5 Optional: run the automated tests

The application does not start test tooling during normal use. To run the containerized unit, integration, and browser-driven test suite:

```
docker compose --profile test up -d --wait selenium
docker compose --profile test run --rm test
```

A successful run ends with PHPUnit reporting no failures or errors. The browser tests use a separate Selenium container and may take longer on the first run.

Stop the application and test services when finished:

```
docker compose --profile test down
```

The database volume is retained unless `-v` is added.

6 Troubleshooting

6.1 Docker cannot connect

If a command reports that the Docker daemon is unavailable, start Docker Desktop or the Linux Docker service and retry `docker compose ps`.

6.2 Port 8080 is already allocated

Use the different-port procedure in Section 2.3. If a previous copy of this project is still running, `docker compose down` from its repository folder may release the port.

6.3 The database is unhealthy after an update

The project deliberately uses `mysql:latest`. MySQL may reject an existing data volume after a major-version jump. Inspect the database log:

```
docker compose logs db
```

If the log reports an unsupported in-place upgrade, use the reset procedure in Section 4.4. This deletes and reconstructs the local baseline database.

6.4 The page does not load or cases do not appear

Check the service state and health endpoint:

```
docker compose ps
curl http://127.0.0.1:8080/api/health
docker compose logs app
docker compose logs bootstrap
```

If you selected a different port, replace 8080. A failed bootstrap prevents the application from starting because the baseline is a required dependency; the bootstrap log contains the cause.

6.5 A published image is unavailable on this computer

Run `docker compose build` and start the stack again. This builds for the host environment from the checked-out source and is the preferred fallback for image-architecture or registry-access errors.

6.6 The browser tests cannot start

Confirm that Docker has enough memory available and that ports 4444 and 7900 are not already occupied. The normal learner-facing application does not need these ports; they belong only to the optional Selenium service.

7 Data, privacy, and scope

- All bundled cases are synthetic teaching cases.
- The current interface has no account system and does not persist learner submissions or attempt history.

- The database volume holds the versioned prototype catalogue, case, rule-support, and reference-response baseline.
- The implemented catalogue is a small, traceable subset of the Austrian ICD-10 BMASGPK 2026 material, not the entire catalogue.
- Results express the prototype’s declared rules and case relationships; they are not medical or official coding advice.

8 Further project documentation

This guide covers installation and day-to-day use. The repository contains the following deeper references:

- `docs/IMPLEMENTATION_SPECIFICATION.md` — exact implemented schema, API, build, and deployment contracts;
- `docs/DEVELOPMENT_DOCUMENTATION.md` — technology and design rationale;
- `docs/REQUIREMENTS_TRACEABILITY.md` — requirement-to-evidence mapping;
- `docs/CHANGELOG.md` — dated implementation and documentation history; and
- `HANDOFF.md` — current project snapshot and remaining work.

The editable source for this document is `docs/USER_GUIDE.tex`; the compiled edition is `docs/USER_GUIDE.pdf`. Both are maintained in the repository so the readable PDF and its source can be reviewed together.

A Command summary

Command	Purpose
<code>docker compose pull</code>	Download published images.
<code>docker compose up -d -wait app</code>	Start the application and its dependencies.
<code>docker compose ps</code>	Show service state.
<code>docker compose logs app</code>	Show application logs.
<code>docker compose down</code>	Stop containers; keep the database volume.
<code>docker compose down -v</code>	Stop containers and delete the database volume.
<code>docker compose build</code>	Build images from the local checkout.
<code>docker compose -profile test run -rm test</code>	Run the full automated test suite after Selenium is started.